

CNW 2140 Computer Networks

Lab 4: build a DNS resolver

Lab handout, version 2. Issued Friday 9 October 2026. This version replaces version 1 of Monday 5 October. Section 10 says which sections changed.

Prepared by the CNW 2140 lab team and approved by Dr. Oleska Brantigern.

1. What you build

A command-line resolver called `tinyresolve`. Given a host name, it finds the address the long way round, with no help: it asks the lab root, follows each referral down the tree, and stops when a server hands it an address record. It prints the address and the servers it asked, in order.

You may write it in Python 3 or in Go. Your program builds and reads DNS messages byte by byte. Library calls that resolve a name for you, or that pack and unpack DNS messages, are not allowed. Sockets and the standard byte-handling modules are.

2. The lab tree

Your resolver talks only to the lab's private DNS tree. The tree has its own root, one top-level zone called `cnwlab`, and three zones under it: `alder.cnwlab`, `birch.cnwlab` and `cedar.cnwlab`. None of these names exists outside the lab.

- The address of the lab root is in the file `root.hint` in the starter repository. Read it from the file; do not type it into your code.
- The tree answers only on the lab subnet. Work in the lab room or over the lab VPN.
- Address records in the lab tree carry a TTL of 40 seconds, so that you can watch a cached answer run out without a long wait.
- The sandbox resolver from the week 3 lecture notes is useful for comparison: ask it for the same name and see whether your answer agrees. Your own resolver must not send its queries there, or to the campus resolver. The grader records where your packets go.

3. The starter repository

Clone `cnw2140-lab4-starter` from the course site. It holds:

- `root.hint`, the address of the lab root.
- `names.txt`, the published list of names your resolver will be given.
- `check_m1` to `check_m4`, one script for each milestone.
- `README`, with the exact output format and two sample runs.
- `capture.sh`, which starts the capture tool with the right filter for the lab subnet.

Do not change the scripts. If one of them looks wrong, say so on the forum.

4. Milestones

Work in this order. Each script tells you whether you have reached its milestone.

- M1: build one query, send it to the lab root over UDP, and print the referral that comes back.
- M2: follow referrals until a server returns an address record.

- M3: follow an alias. When the answer is a CNAME, look up the name it points at, for a chain of up to three aliases.
- M4: keep a cache. Serve a repeated lookup from the cache while the record's TTL has not run out, and go back to the tree once it has.

5. Rules your resolver must keep

- Every lookup finishes within 3 seconds, measured by the grader from the moment it starts your program to the moment the answer is printed. A lookup that runs longer scores nothing for that name.
- Wait at most 750 ms for any one server. If it stays silent, move on to the next server named in the referral. Do not try the same server more than twice.
- Give up after 9 referrals and print FAIL with the reason. A resolver that loops is worse than one that stops.
- Print one line for each lookup in the format shown in the README. The grader compares text, so do not add words of your own.

A line of output looks like this:

```
www.alder.cnwlab 10.40.9.118 via root, cnwlab, alder (cold)
```

6. Truncated replies

No name in the published list or in the hidden list returns a reply large enough to be cut short. You do not need to implement the retry over TCP for this lab, and it is not marked.

7. The grader

- The published list, names.txt, holds 15 names spread over the three zones. It includes two aliases and one name that does not exist.
- The hidden list is a different set of names in the same three zones. We do not say how many names it holds or what they are.
- The grader looks every name up twice: once with a cold cache, and once straight afterwards. For the names it uses to check expiry it then waits for the record to run out and asks a third time.
- The grader runs on the lab machines, not on your laptop. Try your resolver there before you tag.

8. What you hand in

- Your repository, tagged lab4-final. The tag is what gets marked, not the tip of the branch.
- A report of no more than three pages: what your resolver does on a cold lookup of one published name of your choice, shown with your own capture; how your cache decides that a record has run out; and one thing that went wrong and how you found it.

The due date, the hand-in time and the late rules are those already published for the course's labs. This handout does not change them.

9. Marking

Part	Share of the lab mark
Published names answered correctly	35%
Hidden names answered correctly	25%

Part	Share of the lab mark
Cache behaviour, milestone M4	15%
Report	25%

Names are marked one at a time. A name that fails on the cold lookup still earns part of its share if the cached lookup is right, and the other way round.

10. Changes since version 1

- Section 1: a second language is allowed.
- Section 5: the time limit for a lookup has changed.
- Section 7: the published list is longer.
- Section 8: the report is shorter.
- Section 9: the shares for hidden names and for cache behaviour have changed.

Work begun under version 1 needs checking against sections 5 and 7.

11. Mistakes we see every year

- Sending the next query to the server named in the referral without first finding that server's address in the additional section.
- Reading a compressed name as if it were written out in full. Names in replies point backwards into the message.
- Not checking that the ID of a reply matches the ID of the query that was sent.
- Caching a record for ever, or caching it against the wrong name when an alias was followed.
- Printing debugging text to standard output, where the grader reads it as part of the answer.

12. Getting help

Post on the course forum under the tag lab4 so that everybody sees the reply. Bring code problems to the lab drop-in or to the lab session. Please do not email screenshots of code.